FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE

La Clase 10 introduce al estudiante al ecosistema moderno del desarrollo front-end basado en frameworks, una de las áreas de mayor evolución en la ingeniería de software actual. El objetivo central es comprender la transición histórica desde enfoques tradicionales basados en manipulaciones directas del DOM (como jQuery) hacia arquitecturas declarativas basadas en componentes reutilizables, estados reactivos, routers integrados y ciclos de vida bien definidos, como los que ofrecen React, Vue, Angular, Svelte o ASP.NET Blazor.

Esta clase establece fundamentos conceptuales indispensables para el desarrollo de interfaces profesionales, escalables y mantenibles. A lo largo de los cinco subtemas se analiza:

1. La evolución del front-end y de qué manera los frameworks transformaron el desarrollo web.
2. El concepto de componente como unidad mínima reutilizable y su relación con el renderizado declarativo.
3. El manejo de propiedades, eventos y estado como eje del comportamiento dinámico de la UI.
4. La estructura típica de un proyecto moderno y los patrones arquitectónicos más importantes.
5. La integración nativa entre frameworks y APIs REST, incluyendo la comunicación asíncrona, manipulación de datos, control de errores y actualización del estado global.

La comprensión profunda de este contenido permitirá que el estudiante construya interfaces modulares y robustas, integradas de manera eficiente con backends modernos, completando así la estructura fundamental de una plataforma digital contemporánea.

Clase 10: 10. Front-end moderno con frameworks.

RDA 2: Programar aplicaciones web basadas en el estilo arquitectónico cliente-servidor hasta su publicación, garantizando la eficiencia y la seguridad en la comunicación entre el cliente y el servidor, complementado su desarrollo con IA.

Reto # 2

10.1. Evolución del desarrollo front-end: de jQuery a React/Vue/Angular/ASP.NET

La evolución del desarrollo front-end es uno de los procesos más dinámicos e influyentes en la historia de la ingeniería de software. Desde los primeros años del web (1995–2005) hasta la consolidación de frameworks modernos como React, Vue, Angular o ASP.NET, el desarrollo del lado del cliente ha pasado de ser una tarea limitada a simples interacciones visuales, a convertirse en una disciplina compleja basada en arquitecturas modulares, componentes reutilizables, gestión avanzada del estado y estrecha

integración con APIs distribuidas. Comprender esta evolución permite al estudiante entender por qué hoy se requieren frameworks avanzados y cómo estos solucionan problemas que, en épocas anteriores, eran difíciles de manejar con herramientas básicas como jQuery o JavaScript puro.

1. Los orígenes: HTML, CSS y JavaScript (1995–2005)

En esta etapa el front-end se caracterizaba por ser estático. Las páginas estaban compuestas principalmente de HTML para estructura, CSS para estilo y JavaScript para manejar eventos simples como clics o validaciones de formularios. Aunque estos lenguajes siguen siendo esenciales hoy, en esa época no existían principios como modularidad, patrones arquitectónicos, componentes o manejo del estado.

JavaScript carecía de estándares claros y cada navegador implementaba su propia versión. La manipulación del DOM era extensa y verbosa:

```
document.getElementById("btn").addEventListener("click", function () {
  alert("Hola");
});
```

A medida que las páginas crecían, se volvía difícil mantener el código, ya que todo se manejaba en un solo archivo gigante lleno de funciones globales. No existía separación de responsabilidades. Cualquier proyecto grande se volvía inmanejable.

2. jQuery: la primera gran revolución (2006–2015)

La aparición de jQuery marcó un antes y un después en el desarrollo front-end. Su lema "**Write less, do more**" (Escribe menos, haz más) captura su esencia: simplificar la manipulación del DOM, eventos y peticiones AJAX, además de resolver los problemas de compatibilidad entre navegadores que por años afectaron a los desarrolladores.

Ejemplo:

```
$("#btn").click(() => alert("Hola"));
```

Esto reducía varias líneas a una sola, lo cual permitió que miles de desarrolladores pudieran crear interfaces dinámicas de manera rápida.

Sin embargo, jQuery tenía limitaciones estructurales:

Tabla 1. Problemas de jQuery

Problema	**Descripción**
Manipulación manual del DOM	Propensa a errores y difícil de optimizar
Falta de componentes	El código se repetía en muchas partes
No había arquitectura formal	Difícil escalar proyectos grandes
Manejo de estado rudimentario	No había forma nativa de sincronizar vistas con datos

Esto hacía que los proyectos medianos y grandes se volvieran caóticos, especialmente cuando aparecieron aplicaciones web más complejas como Gmail o Facebook.

3. Nacimiento del enfoque SPA (Single Page Application)

El concepto de SPA surge como respuesta a la necesidad de construir aplicaciones web interactivas, rápidas y sin recargas completas, parecidas a aplicaciones móviles o de escritorio. Una SPA carga una sola página HTML y luego, mediante JavaScript, actualiza el contenido dinámicamente.

Esto exigió nuevas herramientas para manejar:

- Rutas internas sin recargar página
- Estados complejos
- Componentes reutilizables
- Comunicación permanente con APIs
- Renderizado eficiente

jQuery ya no era suficiente. Era necesario adoptar arquitecturas más robustas. Ahí es donde surgen frameworks completos como AngularJS y, posteriormente, React y Vue.

4. AngularJS: el primer framework SPA masivo (2010–2016)

AngularJS, desarrollado por Google, fue el primer framework en ofrecer una solución integral para crear SPA. Introdujo conceptos avanzados para la época:

- **Two-Way Data Binding** (sincronizar automáticamente modelo ↔ vista)
- **Directivas** para extender HTML
- **Inyección de dependencias**

- **Controladores** para manejar lógica

AngularJS permitió crear aplicaciones dinámicas, pero también tenía una curva de aprendizaje alta y problemas de rendimiento conforme crecía el proyecto.

5. React: el cambio de paradigma (2013–actualidad)

React, creado por Facebook, revolucionó por completo el front-end con dos ideas fundamentales:

5.1. El Virtual DOM

React no manipula el DOM directamente; en cambio, construye una representación virtual y calcula los cambios mínimos necesarios para actualizar la interfaz:

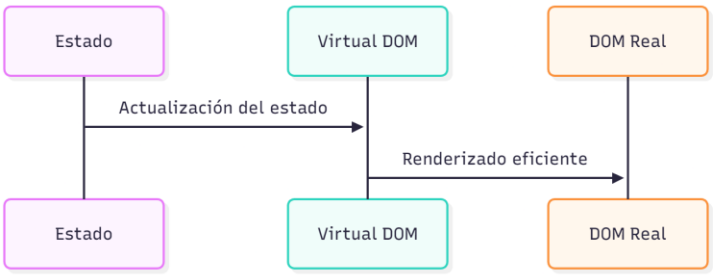


Figura 1. Secuencia estado – virtual DOM – DOM real

Este mecanismo incrementa significativamente el rendimiento de aplicaciones complejas.

5.2. Programación declarativa

En lugar de decir “cómo” cambiar el DOM, React permite declarar “qué” debe verse en función del estado, haciendo el código más sencillo, predecible y fácil de depurar.

5.3. Componentes como unidades básicas

React establece que todo en la UI es un componente:

- botones
- listas
- tablas
- formularios

Esto lleva a una arquitectura modular, reutilizable y fácil de mantener.

6. Vue.js: la alternativa progresiva (2014–actualidad)

Vue se creó combinando lo mejor de Angular y React:

- HTML declarativo con plantillas (Angular)
- Componentes reactivos (React)
- Simplicidad y curva de aprendizaje muy suave

Vue es preferido por equipos pequeños, startups o proyectos que requieren rapidez sin sacrificar arquitectura.

7. Angular moderno (Angular 2+): el estándar empresarial

Angular fue completamente reescrito en 2016. Entre sus características:

- Uso obligatorio de TypeScript
- Arquitectura robusta basada en módulos, servicios, componentes
- CLI muy poderosa
- Estandarización estricta → ideal para empresas

Tabla 2. Comparación de enfoques:

Framework	**Filosofía**	**Dificultad**	**Uso ideal**
React	Biblioteca para componentes	Media	Proyectos flexibles
Vue	Framework progresivo	Baja	Equipos pequeños
Angular	Framework completo	Alta	Empresas grandes

8. ASP.NET Razor y Blazor: el enfoque desde C#

Microsoft también aportó alternativas front-end orientadas a ecosistemas .NET:

Razor Pages

- Renderizado desde el servidor
- Útil para aplicaciones tradicionales

Blazor

- Permite escribir código front-end en C# en lugar de JavaScript
- Usa WebAssembly para ejecutar .NET en el navegador
- Filosofía similar a React + C#

9. Línea histórica completa

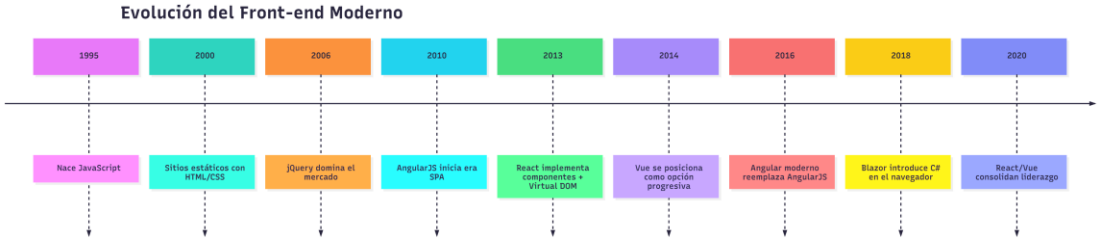


Figura 2. Evolución del front-end

10. Conclusión

La evolución del front-end no es solo una sucesión de herramientas; es un cambio radical en la manera de pensar sobre interfaces, arquitectura y experiencia del usuario. Desde la manipulación manual del DOM hasta la composición declarativa basada en componentes, la historia muestra un proceso continuo de simplificación, abstracción, organización y optimización.

Hoy, React, Vue, Angular y ASP.NET representan la madurez del desarrollo web moderno, cada uno con una filosofía distinta, pero con un propósito común: permitir construir aplicaciones dinámicas, escalables y conectadas a APIs, que respondan a las demandas actuales de rendimiento, mantenibilidad y complejidad.

10.2. Concepto de componente y renderizado

El concepto de componente es uno de los pilares fundamentales del desarrollo front-end moderno. Tanto React, Vue, Angular, Svelte, Blazor y la mayoría de las tecnologías contemporáneas se basan en la idea de que una interfaz debe construirse como un conjunto de piezas autónomas, reutilizables y organizadas, en lugar de ser un único documento HTML manipulado manualmente. Esta visión ha transformado la forma de pensar las interfaces, haciéndolas más escalables, mantenibles y fáciles de probar.

1. ¿Qué es un componente?

Un componente es una unidad independiente de una interfaz que cumple tres principios:

1. **Encapsula estructura (HTML), comportamiento (JS) y estilo (CSS):** Todo lo necesario para su funcionamiento está dentro del componente.
2. **Es reutilizable:** Puedes usarlo varias veces en diferentes partes de la interfaz.
3. **Es autónomo en su estado y lógica:** El componente sabe cómo comportarse, cómo renderizarse y cómo reaccionar a eventos.

Ejemplos típicos de componentes:

- botones
- tarjetas
- formularios
- barras de navegación
- listas de productos
- modales
- tablas paginadas
- gráficos

En frameworks como React, un componente puede verse así:

```
function CardProducto({ nombre, precio }) {
 return (
   <div className="card">
     <h3>{nombre}</h3>
     <p>${precio}</p>
   </div>
 );
}
```

Pero más allá de la sintaxis, lo importante es la organización mental del proyecto.

2. Composición de componentes

La composición es la idea de que una interfaz se arma combinando componentes pequeños para formar otros más grandes, como si fuera LEGO.

Ejemplo conceptual:

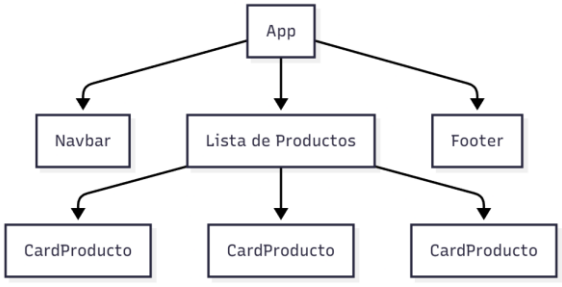


Figura 3. Composición de componentes

Cada bloque es independiente, pero juntos forman la aplicación completa.

Esto permite:

- dividir un sistema en piezas pequeñas
- mantener código limpio y organizado
- evitar duplicación
- reemplazar componentes sin afectar al resto
- facilitar la escalabilidad

3. El ciclo de renderizado

El renderizado es el proceso mediante el cual un framework genera la interfaz visible en el navegador. Los frameworks modernos siguen un ciclo:

1. El componente recibe props y estado.
2. El framework interpreta esa información.
3. Se genera una representación interna (Virtual DOM o mecanismo equivalente).
4. Se actualiza el DOM real de forma optimizada.

Este proceso es automatizado. El desarrollador ya no manipula manualmente el DOM (como con jQuery).

En React, por ejemplo:

```
UI = f(estado)
```

Si el estado cambia, el UI cambia automáticamente.

Esto conduce al concepto de programación declarativa: el desarrollador describe la interfaz y el framework decide cómo actualizarla.

4. Diagrama de secuencia del renderizado reactivo

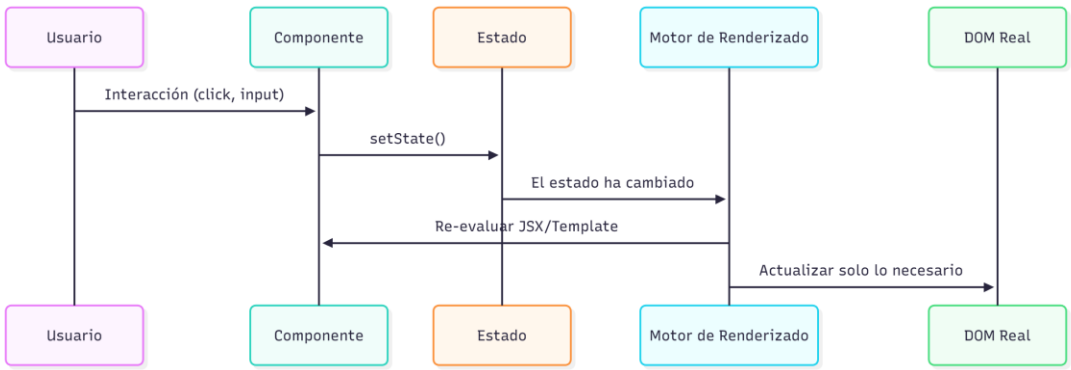


Figura 4. Secuencia renderizado reactivo

Este flujo se repite cada vez que la UI cambia.

5. Tipos de componentes

Los componentes modernos se pueden clasificar en dos grandes categorías:

5.1 Componentes de presentación (presentational components)

- Enfocados en mostrar datos.
- No gestionan lógica compleja ni estado global.
- Reutilizables en varias partes de la interfaz.

Ejemplo: tarjeta de producto, botón, alerta.

5.2 Componentes contenedores (container components)

- Manejan lógica, comunicación con APIs, estado.
- Pasan datos a componentes de presentación.

Ejemplo: vista “Catálogo”, vista “Carrito”.

Tabla 3. Tipos de componentes

Tipo	**Responsabilidad principal**	**Reutilización**	**Manejo de estado**
Presentación	Mostrar UI	Alta	No
Contenedores	Lógica + conexión a API	Media	Sí

6. Props: comunicación hacia abajo

Las props (propiedades) son la forma en que un componente “padre” envía datos a un componente “hijo”.



Figura 5. Ejemplo conceptual

Esto mantiene el flujo de datos unidireccional: de arriba hacia abajo.

7. Estado: datos internos del componente

El **estado (state)** representa valores que cambian con el tiempo y afectan el renderizado.

Ejemplos:

- cantidad seleccionada
- texto en un formulario
- si un modal está abierto
- si un botón está habilitado

En React:

```
const [contador, setContador] = useState(0);
```

Cuando cambias el estado, React vuelve a renderizar el componente automáticamente.

8. Diagrama de estados de un componente

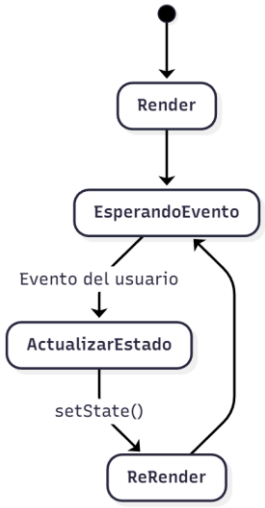


Figura 6. Estados de un componente

Este diagrama muestra cómo el componente entra en un ciclo continuo: renderizar → esperar eventos → actualizar → volver a renderizar.

9. Renderizado condicional

Los componentes pueden renderizar diferentes elementos según su estado.

Ejemplo en pseudocódigo:

```
if (logeado) {
  return <PanelAdmin />
} else {
  return <LoginForm />
}
```

Esto permite crear experiencias dinámicas.

10. Componentes controlados y no controlados

Componentes controlados

- El estado del componente controla el valor del input.

Componentes no controlados

- El DOM maneja el valor; el componente accede solo cuando lo necesita.

Tabla 4. Ventajas y desventajas de los tipos de componentes

Tipo	**Ventaja**	**Desventaja**
Controlado	flujo claro, validaciones potentes	más código
No controlado	simple, directo	menos control

11. Ciclo de vida de un componente

Muchos frameworks tienen métodos o hooks especiales que permiten ejecutar lógica antes o después de ciertos momentos.

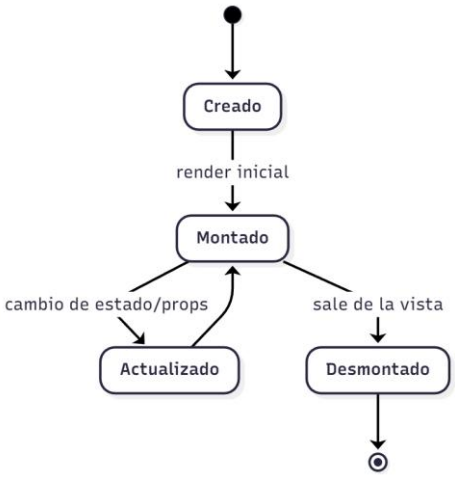


Figura 7. Ciclo de vida de un componente

React usa hooks como:

- useEffect()
- useLayoutEffect()

Vue usa:

- mounted()
- created()

Angular usa:

- ngOnInit()
- ngOnDestroy()

12. Ventajas del enfoque basado en componentes

Tabla 5. Ventajas del enfoque en componentes

Ventaja	**Descripción**
Reutilización	mismos componentes en toda la app
Escalabilidad	proyectos grandes más fáciles de mantener
Independencia	cada componente se prueba o reemplaza sin afectar a otros
Legibilidad	código organizado
Depuración	más fácil de rastrear errores
Mantenibilidad	cada parte tiene su propio archivo

13. Componentización y atomic design

El diseño atómico propone construir componentes como átomos, moléculas y organismos.



Figura 8. atomic design

Los frameworks modernos aplican este enfoque arquitectónico.

14. Conclusión

El concepto de componente es la unidad mínima de construcción en el desarrollo front-end moderno. Gracias a su naturaleza modular y autónoma, permite desarrollar aplicaciones escalables, mantenibles y robustas. El renderizado reactivo, el flujo unidireccional de datos, la composición y el ciclo de vida son pilares que permiten que frameworks como React, Angular, Vue o Blazor puedan crear interfaces altamente dinámicas, respondiendo a los cambios en tiempo real sin necesidad de manipular directamente el DOM.

Damián, cuando desees continuamos con:

10.3. Propiedades, eventos y manejo de estado

El desarrollo front-end moderno se fundamenta en tres conceptos esenciales que permiten la construcción de interfaces dinámicas, modulares y coherentes: propiedades (props), eventos y estado (state). Sin estos conceptos, sería imposible estructurar aplicaciones reactivas, ya que la interfaz no podría responder a cambios en datos, acciones del usuario o actualizaciones provenientes de APIs externas. Este punto de la clase tiene como objetivo desglosar cada uno de estos elementos, explicar cómo interactúan y mostrar cómo los frameworks modernos los implementan de forma eficiente.

1. Propiedades (Props): Datos que fluyen de padres a hijos

Las props son datos que un componente "padre" envía a un componente "hijo". Representan un flujo de información unidireccional, uno de los principios arquitectónicos esenciales de React y adoptado también por Vue, Svelte y Angular (donde se llaman *inputs*).

Conceptos clave:

1. **Inmutabilidad**: las props no deben modificarse dentro del componente hijo.
2. **Unidireccionalidad**: los datos siempre fluyen de arriba hacia abajo (top-down).
3. **Reactividad**: cuando el padre cambia la prop, el hijo se vuelve a renderizar automáticamente.

Ejemplo (independiente del framework)

Un componente padre que envía datos:

```
<ListaProductos productos={lista} />
```

El hijo los recibe:

```
function ListaProductos(props) {
  return props.productos.map(p => <CardProducto data={p} />);
}
```

Tabla 6. Beneficios de las props

Beneficio	**Explicación**
Reutilización	Un mismo componente puede recibir distintos datos.
Predictibilidad	Al no ser mutables, su comportamiento es estable.
Trazabilidad	El flujo de datos es claro y fácil de depurar.

2. Diagrama: Flujo de props en una jerarquía de componentes

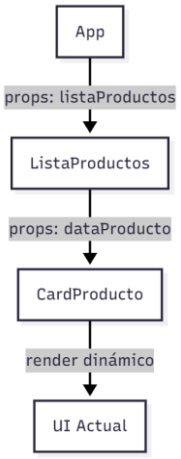


Figura 9. Flujo de props en una jerarquía de componentes

Este diagrama ilustra cómo los datos fluyen desde la raíz de la aplicación hacia los componentes más específicos.

3. Eventos: interacción desde el usuario hacia el sistema

Si las props representan el flujo de datos de **padre → hijo**, los eventos representan el flujo **hijo → padre**.

Los eventos permiten que:

- el usuario interactúe con la interfaz
- los componentes notifiquen cambios
- las acciones disparen actualizaciones de estado
- los sistemas consuman APIs o ejecuten lógica interna

Ejemplos de eventos comunes

- onClick → cuando se hace clic en un botón
- onInput / onChange → cuando cambia un campo de texto
- onSubmit → cuando se envía un formulario
- onMounted o ngOnInit → cuando un componente se monta
- onDestroy → cuando desaparece de la vista

Ejemplo

```
<button onClick={handleComprar}>
  Comprar
</button>
```

Cómo se procesan los eventos

Cuando un evento ocurre:

1. El framework captura la interacción.
2. Ejecuta la función correspondiente.
3. La función puede actualizar el estado o llamar APIs.
4. El cambio de estado produce un re-render automático.

4. Diagrama: Secuencia completa de un evento

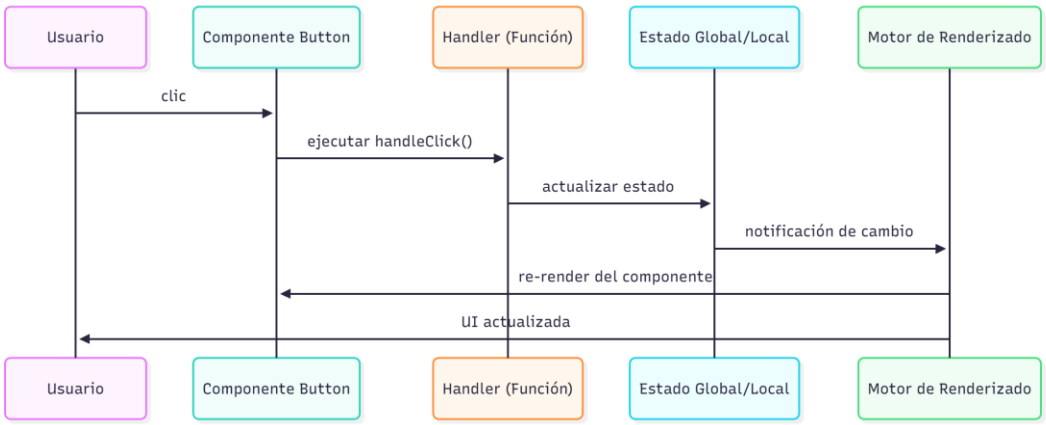


Figura 10. Secuencia completa de un evento

5. Estado (State): el corazón del renderizado reactivo

El estado representa información que cambia con el tiempo y que afecta directamente lo que se muestra en pantalla.

Características principales

1. **Mutable**: a diferencia de las props, el estado sí cambia.
2. **Causa re-render**: cuando cambia, el componente o vista se vuelve a dibujar.
3. **Pertenece al componente**: vive dentro del componente que lo administra.
4. **Controlado por funciones especiales**: como setState o useState.

Ejemplo simplificado

```
const [cantidad, setCantidad] = useState(0);

return (
  <button onClick={() => setCantidad(cantidad + 1)}>
    Agregar ({cantidad})
  </button>
);
```

Cada clic incrementa el contador y el componente vuelve a renderizarse.

6. Diagrama: Estados de un componente

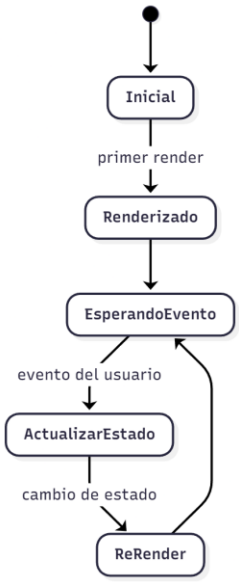


Figura 11. Estados de un componente

7. ¿Por qué necesitamos manejar estado?

El estado permite:

- contabilizar cantidades en un carrito
- mostrar/ocultar elementos
- almacenar datos obtenidos desde API
- manejar filtros, paginación y búsqueda
- cambiar estilos y modos de la UI
- manejar formularios complejos

Sin estado, una aplicación moderna sería estática e incapaz de reaccionar a cambios del usuario o del sistema.

8. Tipos de estado en aplicaciones modernas

Tabla 7. Estado en aplicaciones modernas

Tipo	Descripción	Ejemplos
Estado local	Vive dentro de un componente	Modal abierto, input
Estado global	Se comparte entre múltiples componentes	Usuario autenticado
Estado derivado	Valor calculado desde otros estados	Total del carrito
Estado remoto	Datos provenientes de APIs	Productos, pedidos

En aplicaciones empresariales, el manejo de estado puede volverse complejo, por lo que frameworks como React tienen bibliotecas como Redux, Zustand o Context API; Angular usa servicios con RxJS; Vue usa Vuex o Pinia.

9. Patrón unidireccional de datos

Los frameworks modernos aplican un patrón estricto:

State → UI → Event → New State → UI → …

Este ciclo tiene varias ventajas:

- evita inconsistencias
- hace que la UI sea predecible
- simplifica depuración
- permite herramientas automáticas de debugging

10. Relación entre props, eventos y estado

La interacción entre ellos forma el núcleo de las SPA modernas.

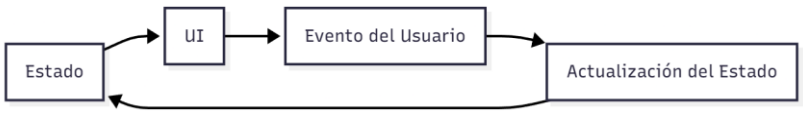


Figura 12. Relación entre props, eventos y estado

Es un ciclo continuo, claro y estructurado.

11. Diferencias entre props y estado

Tabla 8. Diferencias entre props y estado

Característica	**Props**	**Estado**
Quién lo define	Componente padre	El componente mismo
Mutable	No	Sí
Causa re-render	Sí	Sí
Flujo	Padre → hijo	Interno al componente
Propósito	Configurar al componente	Manejar cambios dinámicos

12. Cómo se integran en un proyecto real

En un proyecto típico como el CaféCat SPA que desarrollaste con tus estudiantes:

- Las props se usan para pasar datos a tarjetas de productos.
- Los eventos se disparan al hacer clic en “Agregar al carrito”.
- El estado se usa para almacenar los productos agregados.
- Al cambiar el estado, la UI del carrito se vuelve a renderizar automáticamente.
- El panel admin también usa estado para mostrar los pedidos cargados desde la API.

13. Conclusión

La combinación de props, eventos y estado constituye el fundamento profundo del front-end moderno. Este modelo permite construir aplicaciones reactivas donde:

- los datos fluyen con orden,
- la interfaz refleja siempre el estado actual,
- los usuarios interactúan generando cambios en tiempo real,
- los componentes se mantienen aislados, cohesivos y reutilizables.

Comprender este triángulo conceptual es esencial para dominar frameworks como React, Vue, Angular y Blazor, y para construir aplicaciones dinámicas que se integren correctamente con APIs REST, microservicios o backends tradicionales.

10.4. Estructura de un proyecto front-end

La estructura de un proyecto front-end moderno es un aspecto fundamental para garantizar mantenibilidad, escalabilidad, orden y eficiencia en el desarrollo de aplicaciones. A diferencia de los primeros años del desarrollo web, donde todo se incluía en un único archivo HTML acompañado de un CSS y un pequeño script JavaScript, los frameworks actuales requieren una organización clara basada en componentes, módulos, servicios, rutas y estados.

Entender esta estructura es esencial para cualquier estudiante que desee trabajar profesionalmente en el desarrollo web, porque todos los frameworks modernos —React, Angular, Vue, Svelte, Next.js, Nuxt, Remix, Astro, Blazor, etc.— comparten principios arquitectónicos.

1. ¿Por qué es necesaria una estructura de proyecto?

A medida que las aplicaciones se volvieron más grandes, surgieron desafíos importantes:

- Código duplicado
- Falta de modularidad
- Difícil colaboración entre varios desarrolladores
- Problemas para escalar a cientos de componentes
- Rutas difíciles de mantener
- Desorden en los archivos

Los frameworks modernos ofrecen convenciones que permiten:

- dividir la aplicación en piezas lógicas
- reutilizar componentes
- separar responsabilidades
- facilitar pruebas automatizadas
- integrar herramientas de construcción (bundlers)
- compilar, minificar y optimizar el código

2. Estructura típica de un proyecto moderno

Aunque cada framework tiene sus particularidades, la estructura general suele ser similar. Este es un ejemplo generalizado:

```
/proyecto/
└── src/
    ├── components/
    ├── pages/ ó views/
    ├── services/
    ├── store/ ó state/
    ├── assets/
    ├── router/
    ├── utils/
    ├── styles/
    ├── App.js / App.vue / App.tsx
    └── main.js / index.js / bootstrap.ts
```

A continuación, se explican estas carpetas.

3. Carpeta src/: el corazón del proyecto

Todos los frameworks modernos separan la lógica en un directorio central llamado **src** (source).
Este contiene todo el código ejecutado por el navegador.

4. components/ – Componentes reutilizables

La carpeta más importante en frameworks modernos es **components**, porque los componentes son las unidades básicas de construcción.

Ejemplos típicos:

- Navbar.vue
- CardProducto.jsx
- BotonPrimario.tsx
- ModalEliminar.vue

Un componente bien diseñado debe ser:

- Autónomo
- Reutilizable
- Cohesivo
- Independiente

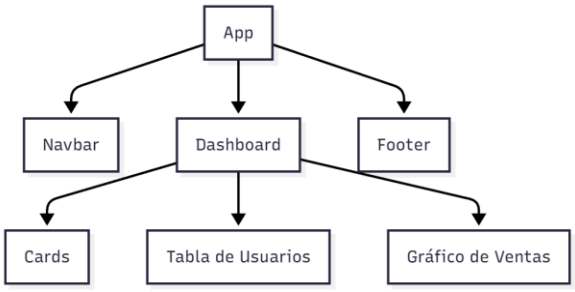


Figura 13. Diagrama de jerarquía de componentes

Esta jerarquía muestra cómo se componen las interfaces modernas.

5. pages/ o views/ – Vistas o pantallas completas

Las "vistas" son componentes de mayor nivel que representan páginas completas:

- Home.vue
- Catalogo.jsx
- Carrito.jsx
- AdminPanel.vue

Aquí se suele manejar:

- lógica de negocio de cada pantalla
- consumo de APIs
- composición de componentes
- manejo de estados globales

6. router/ – Sistema de rutas

Los frameworks modernos incluyen enrutadores para aplicaciones SPA.

- React → React Router
- Vue → Vue Router
- Angular → Router integrado
- Svelte → Routify, SvelteKit

Esta carpeta contiene:

```
router/
  └── index.js
```

Ejemplo conceptual de definición de rutas:

```
{
  path: "/catalogo",
  component: Catalogo
}
```



Figura 14. Diagrama de flujo de navegación

7. services/ – Comunicación con APIs (capa de datos)

Aquí se ubica toda la lógica relacionada con comunicación externa:

- consumo de APIs REST
- autenticación
- CRUD sobre entidades
- manejo de errores
- almacenamiento local

Ejemplo típico:

```
services/
   ├── productos.service.js
   ├── usuarios.service.js
   ├── auth.service.js
   └── pedidos.service.js
```

Esto separa los datos de la interfaz, manteniendo limpios los componentes.

8. store/ o state/ – Manejo de estado global

En proyectos grandes, el estado necesita un lugar centralizado.

Dependiendo del framework:

Tabla 9. Librería de estado de cada framework

Framework	**Librería de estado**
React	Redux, Zustand, Jotai, Context API
Vue	Vuex o Pinia
Angular	NgRx o servicios + RxJS
Svelte	writable/derived stores

El store permite:

- compartir datos entre componentes
- evitar props innecesarias
- sincronizar el estado con la UI
- manejar lógica compleja (auth, permisos, etc.)

9. assets/ – Imágenes, fuentes, íconos y archivos estáticos

Aquí se guardan:

- imágenes (.png, .jpg, .svg)
- fuentes (.ttf, .woff)
- archivos estáticos (.json, .pdf)

Ejemplo:

```
assets/
  ├── logo.svg
  ├── banner.jpg
  ├── icons/
  └── estilos-globales.css
```

10. styles/ – Estilos globales

Aunque los componentes generalmente tienen sus propios estilos, también existe un archivo general:

```
styles/
    ├──── global.css
    └──── variables.css
```

Incluye:

- variables de color

- tipografías
- resets
- estilos compartidos

11. Estructura específica por framework

Cada framework implementa estas ideas con variaciones propias.

11.1 React

```
src/
  ├── components/
  ├── pages/
  ├── context/
  ├── hooks/
  ├── services/
  ├── assets/
  ├── App.jsx
  └── main.jsx
```

React utiliza **JSX**, estados locales con useState, efectos con useEffect y enrutamiento con React Router.

11.2 Vue

```
src/
  ├── components/
  ├── views/
  ├── router/
  ├── store/
  ├── assets/
  ├── App.vue
  └── main.js
```

Vue agrupa HTML, CSS y JS dentro de un mismo archivo .vue.

11.3 Angular

```
src/
  ├── app/
      ├── components/
      ├── services/
      ├── models/
      ├── pages/
      └── app.module.ts
  ├── assets/
  └── environments/
```

Angular organiza todo en **módulos** y usa TypeScript de forma obligatoria.

11.4 ASP.NET Blazor

```
Pages/
Shared/
wwwroot/
Program.cs
App.razor
```

Aquí los componentes son archivos .razor escritos en C#.

12. ¿Cómo se integran estas carpetas en el ciclo de desarrollo?

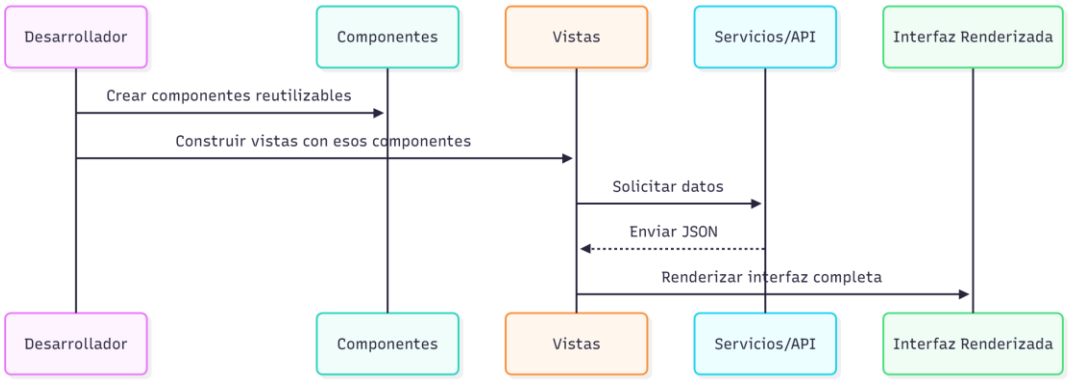


Figura 15. El flujo típico en un proyecto moderno

Este ciclo permite:

- reducción de duplicación
- modularización
- mejores pruebas
- crecimiento ordenado

Tabla 10. Comparación entre arquitecturas antiguas y nuevas

Aspecto	**Web sin frameworks**	**Web moderna (SPA)**
Organización	Archivos sueltos	Estructura modular
Componentes	No existen	Base del sistema
Estado	Variables globales	Sistemas estructurados
Navegación	Recarga completa	Router interno
Reutilización	Mínima	Elevada
Escalabilidad	Baja	Alta

13. Conclusión

La estructura de un proyecto front-end moderno es el resultado de años de evolución, aprendizajes y prácticas recomendadas. Su propósito es facilitar el desarrollo sostenible, escalable y colaborativo de aplicaciones web que interactúan de forma compleja con APIs y servicios backend. Con carpetas dedicadas a componentes, vistas, servicios, rutas, estado y recursos, los frameworks modernos permiten que el código se mantenga limpio, organizado y fácil de ampliar.

Los estudiantes deben comprender que una buena organización del proyecto no es un lujo, sino un requisito indispensable para cualquier desarrollo profesional, especialmente en aplicaciones que crecerán con el tiempo y serán mantenidas por múltiples desarrolladores.

10.5. Integración con datos desde APIs

La integración con APIs es uno de los pilares fundamentales del desarrollo front-end moderno. En cualquier plataforma digital actual —redes sociales, e-commerce, banca digital, servicios de streaming, sistemas empresariales— el frontend no funciona en aislamiento: necesita consumir información que proviene de un backend. Esta comunicación entre el cliente y el servidor es lo que convierte a las aplicaciones web en sistemas dinámicos capaces de responder a datos actualizados en tiempo real.

Este punto de la clase se centra en comprender cómo los frameworks modernos (React, Angular, Vue, Svelte, ASP.NET Blazor, etc.) interactúan con APIs REST, qué patrones se emplean para organizar código, cómo se maneja el estado y qué buenas prácticas se recomiendan para garantizar seguridad, eficiencia y escalabilidad.

1. ¿Qué significa integrar un frontend con una API?

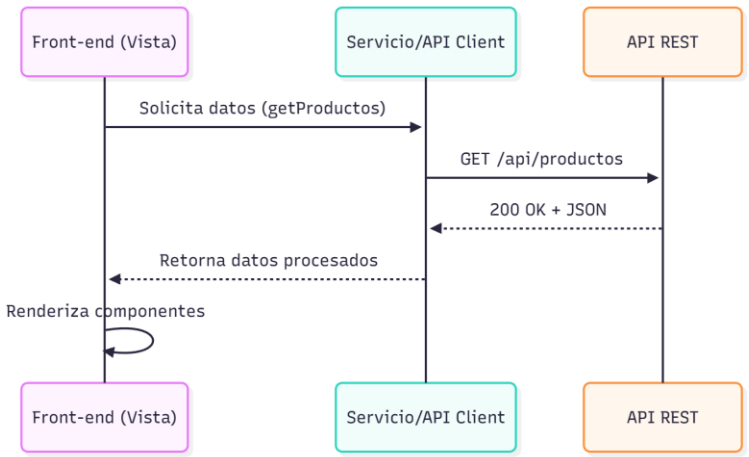
Una API (Application Programming Interface) es un puente que permite que dos sistemas se comuniquen. En el contexto web, la API más común es una API REST basada en HTTP que intercambia datos en formato JSON.

La integración implica:

- Enviar peticiones HTTP desde el frontend.
- Recibir respuestas del backend.
- Transformar esos datos en elementos visuales (components).
- Manejar errores, tiempo de espera, expiración de sesión, permisos y estado global.
- Organizar esta comunicación en servicios reutilizables.

En otras palabras: la API provee los datos; el frontend los muestra y permite interactuar con ellos.

2. Flujo básico cliente–API–UI



Este flujo se repite para cualquier operación CRUD.

Figura 16. Flujo básico fron-end-API

3. Pasos para integrar una API en un framework moderno

Paso 1. Crear un servicio (API Client)

Todos los frameworks modernos recomiendan crear una carpeta como:

```
services/
    ├── productos.service.js
    ├── auth.service.js
    └── pedidos.service.js
```

Aquí se centralizan:

- URLs
- llamadas HTTP
- manejo de errores
- transformación de datos
- autenticación

Ejemplo:

```
export async function getProductos() {
  const res = await fetch("https://api.misitio.com/productos");
  return res.json();
}
```

Paso 2. Consumir el servicio desde un componente

En React, por ejemplo:

```
useEffect(() => {
  productosService.getProductos().then(setProductos);
}, []);
```

En Vue:

```
onMounted(async () => {
  productos.value = await getProductos();
});
```

En Angular:

```
this.productosService.getProductos().subscribe(data => {
  this.productos = data;
});
```

Paso 3. Renderizar datos en la UI

La información recibida se transforma en una lista de tarjetas, tablas o gráficas. La UI se vuelve "reactiva": cambia automáticamente cuando llegan nuevos datos o cuando el usuario interactúa.

Paso 4. Manejar errores y estados de carga

El frontend debe contemplar:

- estado "loading…"
- error de red
- error de autorización (401/403)
- falta de datos

Ejemplo:

```
if (loading) return <Spinner />;
if (error) return <ErrorMessage />;
```

Paso 5. Autenticación y autorización

Al consumir la API, es necesario enviar:

- tokens JWT
- headers como x-user-id y x-user-role
- cookies seguras

Esto permite que el servidor distinga entre usuarios, clientes y admins.

4. Patrones arquitectónicos para consumo de APIs

4.1 Service Layer Pattern

El frontend tiene una capa de servicios dedicada exclusivamente a comunicar con la API.

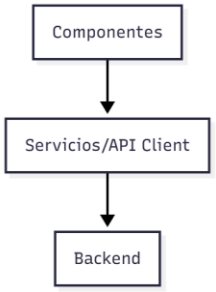


Figura 17. Service Layer Pattern

Beneficios:

- El componente no conoce la URL exacta.
- Se reutiliza la lógica para múltiples vistas.
- Facilita pruebas automatizadas.
- Permite reemplazar el backend sin tocar la UI.

4.2 State Management Pattern

Frameworks modernos utilizan un estado global compartido para almacenar:

- usuario actual
- productos cargados
- carrito de compras
- sesión activa



Figura 18. State Management Pattern

4.3 Presentational vs Container Components

- **Container components:**

  manejan datos, fetch, lógica de negocio.

- **Presentational components:**

  solo reciben props y muestran información.

Ejemplo:

```
pages/Catalogo.jsx   ← maneja API
components/Card.jsx  ← muestra un producto
```

5. Tipos de peticiones que se integran

Los frontends modernos consumen APIs mediante los métodos HTTP:

Tabla 11. Consumo de APIS mediante métodos HTTP.

Acción	**Método HTTP**	**Ejemplo en una tienda**
Leer	GET	Obtener productos
Crear	POST	Registrar un pedido
Actualizar	PUT/PATCH	Editar un producto
Borrar	DELETE	Eliminar usuario

Cada llamada debe manejar correctamente el estado HTTP.

6. ER básico del flujo de datos entre frontend y backend

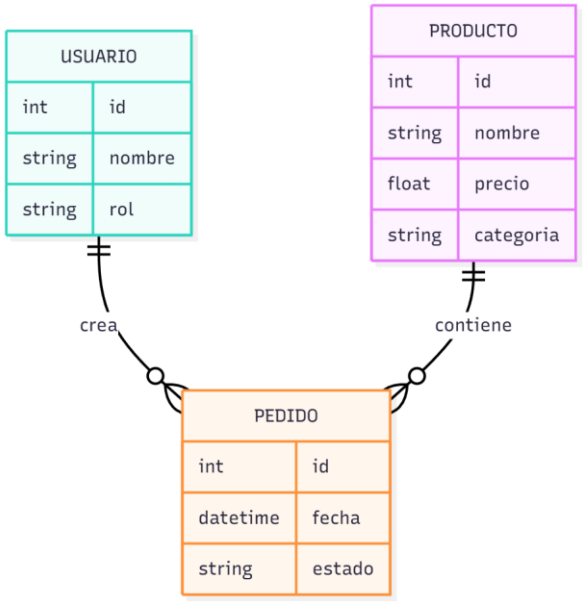


Figura 19. ER del flujo de datos entre frontend y backend

Este diagrama facilita comprender qué entidades viajan entre frontend y backend.

7. Buenas prácticas en integración con APIs

Tabla 12. Buenas prácticas de integración con APIS

Práctica	**Descripción**
Centralizar fetch en servicios	No llamar APIs desde los componentes directamente.
Controlar estados de carga	Evitar interfaces “congeladas”.
Reintentar peticiones automáticas	Útil en redes móviles o inestables.
Agregar logs de diagnóstico	Facilita depuración en producción.
Manejar expiración de sesión	Detectar 401 y redirigir al login.
No exponer datos sensibles	Validación del lado del servidor.
Evitar duplicación de datos	Manejar cache en store global.
Utilizar DTOs o interfaces	Para garantizar formatos consistentes.

8. Ciclo completo de integración con API

El siguiente diagrama de secuencia muestra cómo un usuario agrega un producto al carrito y envía el pedido a la API:

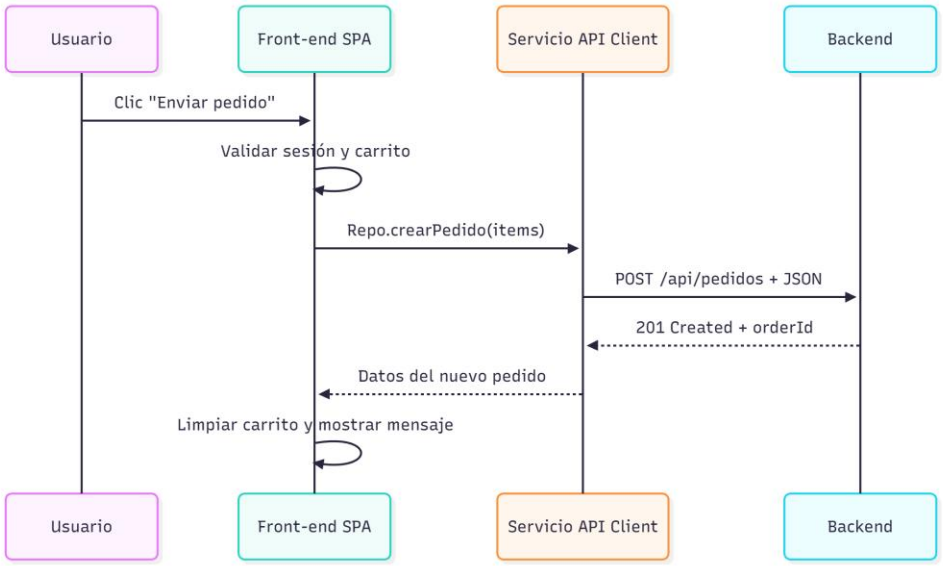


Figura 20. Ciclo de una API

9. Diferencias de integración según el framework

Tabla 13. Características de integración con APIS por framework

Framework	**Características de integración con API**
React	Usa hooks (useEffect, useQuery), fetch o Axios.
Vue	Usa onMounted, Composition API, Vuex/Pinia.
Angular	Cliente HTTP nativo basado en Observables.
Svelte/SvelteKit	Stores reactivos; endpoints propios.
Blazor (C#)	HttpClient integrado y tipado fuerte.

Referencias citadas en la Clase 10.

- Bierman, G., Abadi, M., & Torgersen, M. (2014). *Understanding TypeScript*. Microsoft Research.
- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- Google Developers. (2024). *Angular Documentation*. https://angular.io
- Facebook Open Source. (2024). *React Documentation*. https://react.dev
- Vue.js Team. (2024). *Vue.js Guide*. https://vuejs.org

- Mozilla Developer Network. (2024). *Web APIs and DOM Documentation*. https://developer.mozilla.org
- Svelte. (2024). *Svelte Documentation*. https://svelte.dev/docs
- Microsoft. (2024). *ASP.NET Blazor Documentation*. https://learn.microsoft.com/aspnet/core/blazor.

Definición de términos

- **API Client.** Módulo o servicio dentro de una aplicación front-end encargado de gestionar las solicitudes HTTP hacia un backend remoto.
- **Binding.** Proceso mediante el cual un dato de la aplicación se enlaza con un elemento de la interfaz para que se actualice de forma automática.
- **Component-based Architecture.** Paradigma de diseño front-end que organiza la interfaz en unidades independientes, reutilizables y encapsuladas llamadas componentes.
- **DOM Virtual (Virtual DOM).** Representación en memoria del DOM real que permite actualizaciones eficientes mediante algoritmos de reconciliación.
- **Estado (State).** Conjunto de datos que determinan el comportamiento dinámico de un componente o aplicación; los frameworks lo gestionan de forma reactiva.
- **Eventos (Events).** Acciones producidas por el usuario o el sistema, tales como clics, entradas de texto, cambios de formulario o cargas de datos.
- **Framework Front-end.** Biblioteca estructurada que ofrece herramientas, patrones y APIs para construir aplicaciones de interfaz complejas y reactivas.
- **Renderizado Declarativo.** Técnica donde el desarrollador indica *qué* desea mostrar y es el framework quien determina *cómo* actualizar la UI.
- **Router.** Módulo que administra la navegación interna de una SPA mediante rutas lógicas y parámetros.
- **SPA (Single Page Application).** Arquitectura de aplicaciones web que operan dentro de una sola página HTML, actualizando su contenido de forma dinámica mediante JavaScript.

- **Recursos de profundización**

Recurso: *Profundizacion1_clase9.docx*

ACTIVIDAD DE EVALUACIÓN:

INSTRUCCIONES	*Responde las siguientes preguntas de opción múltiple relacionadas con los temas abordados en la Clase.* *Selecciona la alternativa correcta en cada caso. Al finalizar, revisa la retroalimentación para reforzar tu comprensión.*

Titulo	Front-end moderno con frameworks					
Dificultad		Baja	X	Media		Alta
RDAs Evaluados:		RDA 1				
	X	RDA 2				
		RDA 3				
Secciones Evaluadas		Clase: 10	10. Front-end moderno con frameworks			
PREGUNTA 1	¿Cuál es la principal característica común entre React, Vue y Angular?					
CORRECTA	Son frameworks/librerías basados en componentes reutilizables.					
Incorrecta	Todos utilizan exclusivamente plantillas HTML sin JavaScript.					
Incorrecta	Funcionan únicamente para aplicaciones móviles.					
Incorrecta	Requieren siempre un servidor dedicado para ejecutarse.					
Retroalimentación de las respuestas	Los frameworks modernos comparten el enfoque de arquitectura basada en componentes reutilizables, encapsulando UI, lógica y estilo en unidades independientes.					
PREGUNTA 2	¿Qué concepto describe mejor el *estado* en un framework moderno?					
CORRECTA	El conjunto de datos que determinan la representación dinámica del componente.					
Incorrecta	Los estilos CSS que usa un componente.					
Incorrecta	Las rutas disponibles en la aplicación.					
Incorrecta	Los eventos de teclado del usuario.					
Retroalimentación de las respuestas	El estado define los datos que cambian con el tiempo. Cuando el estado cambia, el framework vuelve a renderizar automáticamente la UI.					
PREGUNTA 3	¿Para qué sirve un *servicio* (service) dentro de un proyecto front-end?					

CORRECTA	Para encapsular llamadas HTTP y separar la lógica de acceso a datos del componente.
Incorrecta	Para almacenar exclusivamente imágenes y archivos estáticos.
Incorrecta	Para definir estilos globales.
Incorrecta	Para gestionar la navegación entre vistas.
Retroalimentación de las respuestas	Los servicios agrupan funciones de comunicación con APIs, permitiendo que los componentes permanezcan limpios y enfocados solo en la UI.
PREGUNTA 4	¿Qué ventaja ofrece una SPA frente a un sitio tradicional basado en múltiples páginas?
CORRECTA	Reduce recargas completas, permitiendo transiciones más rápidas y fluidas entre vistas.
Incorrecta	Elimina completamente la necesidad de un backend.
Incorrecta	Evita el uso de JavaScript.
Incorrecta	Solo funciona con Angular.
Retroalimentación de las respuestas	Las SPA actualizan dinámicamente el contenido sin recargar toda la página, proporcionando una experiencia similar a una aplicación de escritorio.

Enlaces externos recomendados

Título del enlace relacionado: React – Documentación Oficial
Descripción del enlace relacionado:
Documentación oficial del framework React, con explicaciones sobre componentes, hooks, renderizado, manejo de estado y patrones de arquitectura.
Enlace: https://react.dev/learn

Título del enlace relacionado: Vue.js – Guía Oficial
Descripción del enlace relacionado:
Guía oficial del framework Vue, explicando su arquitectura reactiva, componentes,

comunicación entre módulos y manejo de estado con Pinia.
Enlace: https://vuejs.org/guide

Título del enlace relacionado: Angular – Documentación Oficial
Descripción del enlace relacionado:
Portal oficial de Angular con tutoriales, arquitectura basada en TypeScript, servicios, inyección de dependencias, módulos, routing y consumo de APIs.
Enlace: https://angular.io/docs

Título del enlace relacionado: ASP.NET Blazor – Microsoft Docs
Descripción del enlace relacionado:
Guía completa de Blazor para el desarrollo de aplicaciones web interactivas con C#, componentes reutilizables y comunicación con APIs REST.
Enlace: https://learn.microsoft.com/aspnet/core/blazor

Fundamentos Teóricos de Angular 19 (Versión Standalone)

Angular es un framework frontend de desarrollo web creado y mantenido por Google. Se caracteriza por ser un framework completo, altamente estructurado, con énfasis en la arquitectura, la escalabilidad, la productividad empresarial y el uso intensivo de TypeScript. A diferencia de bibliotecas como React o Vue, Angular incluye una solución integral que abarca componentes, enrutamiento, servicios, formularios, inyección de dependencias, testing, herramientas de compilación y CLI.

Angular 19 adopta el enfoque standalone, eliminando la necesidad de módulos (`NgModule`) para simplificar el desarrollo, mejorar la carga diferida y hacer más eficiente la arquitectura.

1. Arquitectura general de Angular

Angular sigue una arquitectura basada en componentes, organizados en capas bien definidas:

- Presentación (Components)
- Lógica de negocio (Services)
- Acceso a datos (HTTP / APIs)
- Routing
- Inyección de dependencias (DI)
- Mecanismo de detección de cambios

Tabla 1. Comparación de los pilares de Angular

Pilar	Descripción	Ejemplo
Componentes	Unidad básica visual y lógica UI	`class HomeComponent {}`
Templates	Vistas HTML con data-binding	`<h1>{{ titulo }}</h1>`
Servicios	Lógica reusada, acceso a APIs	`ProductosService`
Inyección de dependencias	Registrar y consumir servicios	`constructor(private prodSrv: ProductosService)`
Routing	Navegación entre vistas	`/productos`, `/crear`
Change Detection	Actualiza el DOM cuando hay cambios	Estrategia Default o OnPush

2. Flujo visual de Angular

Este diagram integra el ciclo de creación de una vista Angular:



3. Componentes en Angular

Un componente es la unidad básica de UI y se compone de:

- Una clase TypeScript
- Un template HTML
- Una hoja de estilos opcional
- Metadatos (decorador `@Component`)

Ejemplo:

```
@Component({
  selector: 'app-productos',
  standalone: true,
  templateUrl: './productos.component.html'
})
export class ProductosComponent {
  titulo = 'Listado de productos';
}
```

El standalone hace que el componente ya no dependa de un `NgModule`.

Tabla 2. Elementos de un componente

Elemento	Descripción
`selector`	Etiqueta HTML que representa al componente
`templateUrl` o `template`	Vista asociada
`styleUrls` o `styles`	Estilos
`imports`	Para standalone; componentes o directivas que usa
Clase TS	Contiene propiedades, métodos y lógica

4. Sistema de Templates y Data Binding

Angular proporciona múltiples formas de enlazar datos:

One-way binding (solo lectura)

```
<h1>{{ titulo }}</h1>
```

Two-way binding (lectura/escritura)

```
<input [(ngModel)]="nombre">
```

Binding a atributos

```
<img [src]="rutaImagen">
```

Binding a eventos

```
<button (click)="guardar()">Guardar</button>
```

5. Directivas estructurales

Permiten modificar el DOM:

Directiva	Uso
`*ngIf`	Mostrar/ocultar elementos
`*ngFor`	Renderizar listas
`*ngSwitch`	Renderizado condicional múltiple

Ejemplo:

```
<tr *ngFor="let p of productos">
  <td>{{ p.nombre }}</td>
</tr>
```

6. Servicios e Inyección de Dependencias (DI)

Angular es uno de los frameworks con un sistema de DI más robusto del mundo.

Un servicio es una clase reutilizable:

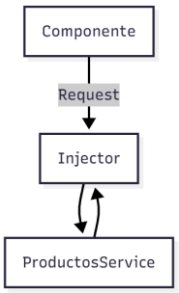
```
@Injectable({ providedIn: 'root' })
export class ProductosService {
  constructor(private http: HttpClient) {}

  listar() {
    return this.http.get('/api/productos');
  }
}
```

La DI permite consumirlo dentro de un componente:

```
constructor(private productosSrv: ProductosService) {}
```

Diagrama DI en Angular



7. Comunicación con APIs – HttpClient

Angular incorpora el módulo `HttpClient` que devuelve **Observable<T>**, lo cual permite:

- peticiones reactivas
- cancelación
- composición funcional (`map`, `filter`, etc.)

Ejemplo:

```
this.http.get<Producto[]>(url)
  .subscribe(prods => this.productos = prods);
```

8. Routing en Angular 19

Con standalone se usa `provideRouter()`:

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'crear', component: CrearProductoComponent },
  { path: '**', redirectTo: '' }
];
```

Y se agrega en `app.config.ts`:

```
providers: [
  provideRouter(routes)
]
```

Diagrama del router

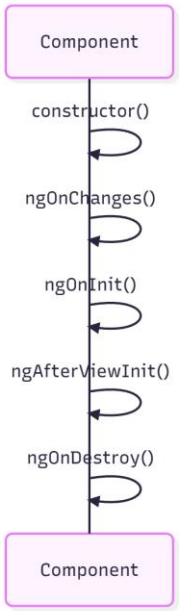


9. Ciclo de vida de un componente

Angular tiene múltiples hooks, pero los más importantes son:

Hook	Momento
`ngOnInit()`	Al iniciar el componente
`ngOnChanges()`	Al cambiar inputs
`ngAfterViewInit()`	Vista inicial ya renderizada
`ngOnDestroy()`	Limpieza

Diagrama del ciclo:



10. Formularios

Angular tiene 2 sistemas:

Formularios reactivos (más usados en producción)

```
form = this.fb.group({
  nombre: ['', Validators.required],
  precio: [0, Validators.min(1)]
});
```

Formularios basados en template

Menos usados en sistemas complejos.

11. Detección de cambios

Angular mantiene actualizado el DOM automáticamente mediante:

- Zone.js (modo clásico)

- `OnPush` (optimizado)

12. Estructura típica de un proyecto

```
src/
  app/
    features/
    services/
    models/
    app.component.ts
  main.ts
  app.config.ts
```

Angular promueve separación clara por responsabilidades.

13. Compilación y despliegue

Angular usa **Angular CLI** y **esbuild** (desde Angular 16+).

Para compilar:

```
ng build --configuration production
```

Se genera:

```
dist/productos-crud-angular/
```

Despliegue típico:

- Firebase Hosting
- Vercel
- Netlify
- AWS Amplify
- Nginx / Apache en un servidor Linux

14. Ventajas de Angular

Ventaja	Razón
Estandarización	Menos “librerías externas”, todo integrado
Escalabilidad	DI + servicios + arquitectura por capas
Robustez	TypeScript obligatorio
Formularios avanzados	El mejor sistema del mercado
CLI poderosa	Generación automática de código
Opinionado	Facilita proyectos grandes

15. Limitaciones de Angular

Desventaja	Por qué
Curva de aprendizaje alta	Muchas piezas: DI, Observables, componentes, routing
Verbosidad	Más código que React/Vue
Pesado para proyectos pequeños	Overkill si la app es pequeña

Fundamentos Teóricos de React 19 + TypeScript (Vite)

React es una biblioteca JavaScript desarrollada por Facebook (Meta), enfocada en la construcción de interfaces de usuario mediante una arquitectura declarativa basada en *componentes funcionales* y *hooks*. A diferencia de Angular, React es una biblioteca, no un framework: esto significa que React se enfoca exclusivamente en la capa de la vista (UI), mientras que todo lo demás —routing, llamadas HTTP, manejo de estado global, testing, estilos, arquitectura— se incorporan mediante librerías externas.

Esta flexibilidad y simplicidad inicial han hecho que React sea el ecosistema más popular del mundo frontend, especialmente en startups y empresas que priorizan rapidez y personalización.

1. Arquitectura general de React

React se basa en tres pilares:

1. Componentes funcionales
2. JSX (JavaScript + XML)
3. Estado y ciclo de renderizado mediante Hooks (`useState`, `useEffect`, etc.)

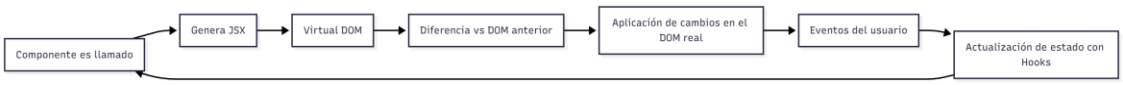
React propone una visión completamente declarativa:

"Describe qué debería mostrar la UI; React se encarga de cómo actualizarla".

Tabla 1. Pilares fundamentales de React

Pilar	Descripción	Ejemplo
JSX	Mezcla JS con HTML en un mismo archivo	`<h1>{titulo}</h1>`
Hooks	Manejo de estado, ciclo de vida y lógica	`useState`, `useEffect`
Virtual DOM	Representación eficiente del DOM	Difing entre árboles virtuales
Renderizado declarativo	Describe el resultado, no el procedimiento	`{cargando && <p>Cargando...</p>}`
Componentes	Funciones puras que devuelven vistas	`function App() { return <h1/> }`

2. Flujo de renderizado de React



3. JSX: La base del desarrollo en React

JSX permite mezclar HTML con JavaScript dentro de una función:

```
function Saludo({ nombre }) {
  return <h1>Hola {nombre}</h1>;
}
```

Ventajas:

- Mejor legibilidad
- Poder expresivo del lenguaje JS
- Más fácil para crear UI dinámicas

4. Componentes funcionales en React

React 19 usa solo componentes funcionales, dejando atrás los "class components".

Ejemplo:

```
function ProductoCard({ producto }) {
  return (
    <div>
      <h3>{producto.nombre}</h3>
      <p>${producto.precio}</p>
    </div>
  );
}
```

Tabla 2. Elementos principales de un componente React

Elemento	Descripción
Props	Datos que entran al componente
Estado	Valores internos que cambian el renderizado
Hooks	Funciones para manejar ciclo de vida y estado
JSX	Sintaxis HTML-like dentro de JS
Pure Function	El componente es una función pura que retorna UI

5. Estado y Hooks

React introdujo los hooks en la versión 16.8, y hoy son el corazón del framework.

5.1 `useState()`

Maneja valores que cambian dentro del componente:

```
const [contador, setContador] = useState(0);
```

5.2 `useEffect()`

Reemplaza todos los ciclos de vida anteriores (`componentDidMount`, `componentDidUpdate`, etc.).

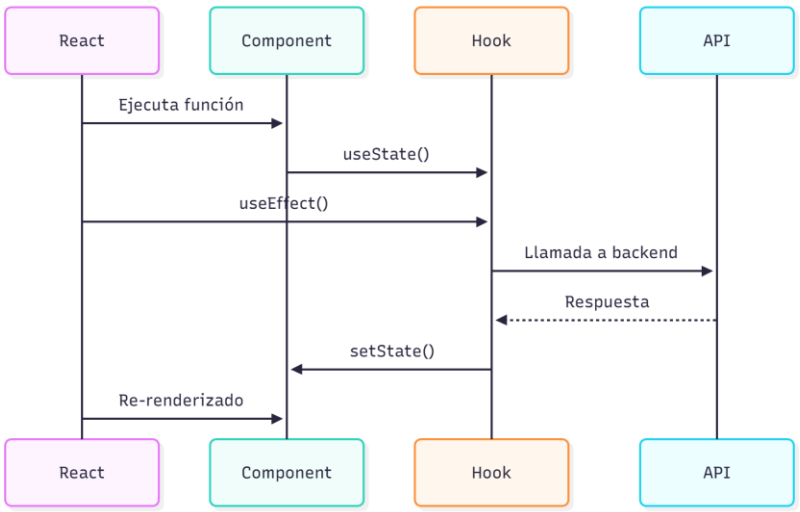
Ejemplo:

```
useEffect(() => {
  cargarProductos();
}, []);
```

5.3 Otros hooks importantes

Hook	Función
`useContext`	Estado global por contexto
`useReducer`	Manejo de estado complejo
`useMemo`	Memorización de valores
`useCallback`	Memorización de funciones
`useRef`	Referencias al DOM

6. Ciclo de vida moderno (Hooks)



7. Props: Comunicación entre componentes

Los componentes reciben información mediante props.

```
<ProductoCard producto={prod} />
```

Dentro del componente:

```
function ProductoCard({ producto }) {
  return <h2>{producto.nombre}</h2>;
}
```

Props son de solo lectura.

8. Lista y renderizado dinámico

React usa `.map()` para iterar listas:

```
{productos.map(p => (
  <tr key={p.id}>
    <td>{p.nombre}</td>
  </tr>
))}
```

9. Manejo de eventos

React sigue la sintaxis camelCase:

```
<button onClick={guardar}>Guardar</button>
```

10. Llamadas HTTP (fetch API)

React no trae módulo HTTP, por lo que se usa:

- `fetch()`
- axios
- o servicios personalizados

Ejemplo:

```
const res = await fetch("/api/productos");
const data = await res.json();
```

React consume promesas, no Observables.

11. Router en React

React Router Dom es la librería estándar:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<ListarProductos />} />
  </Routes>
</BrowserRouter>
```

Para navegar:

```
const navigate = useNavigate();
navigate("/editar/1");
```

Para leer parámetros:

```
const { id } = useParams();
```

12. Estructura típica de un proyecto React

```
src/
  components/
  features/
  services/
  models/
  App.tsx
  main.tsx
  index.css
```

React no impone estructura; esta organización es una buena práctica profesional.

13. Estado global (Redux, Zustand, Context)

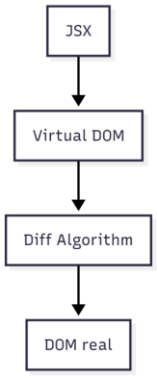
React permite manejar estado global con distintas librerías:

Librería	Uso
Redux Toolkit	Estado global empresarial
Zustand	Ligero y rápido
Context API	Integrado, para estados simples
Jotai	Estado reactivo declarativo

React NO obliga a usar uno; Angular sí impone DI.

14. Virtual DOM

El Virtual DOM es una estructura en memoria que optimiza el renderizado:



Beneficios:

- Mejor rendimiento
- Evita tocar de más el DOM nativo
- Permite animaciones declarativas y reactividad predecible

15. Reglas de React moderno

1. Los componentes deben ser funciones puras.
2. Los hooks no se pueden usar dentro de condicionales.
3. El estado debe ser inmutable.
4. Cada render debe ser determinístico.

Estas reglas permiten a React "predecir" cómo se comporta cada parte de la UI.

16. Paquetes complementarios en ecosistema React

Función	Librerías típicas
Routing	React Router
Estado Global	Redux Toolkit, Zustand
Validación	Yup, Zod
Formularios	Formik, React Hook Form
Estilos	Tailwind, Bootstrap, MUI, Styled Components

17. Ventajas de React

Ventaja	Explicación
Flexibilidad	Tú eliges arquitectura y librerías
JSX poderoso	HTML + JS en un mismo bloque
Rendimiento	Virtual DOM + diff rápido
Popularidad	Mayor ecosistema del mercado
Componentes puros	Reusabilidad y aislamiento
Curva de aprendizaje suave	Fácil para empezar

18. Limitaciones de React

Desventaja	Explicación
No es un framework	Debes integrar libs manualmente
Renderizado excesivo	Si no se optimiza con `memo`
Complejidad en estados grandes	Necesita Redux o Zustand
JSX puede confundir	No es HTML real

19. Despliegue de una app React

Construcción con Vite:

```
npm run build
```

Genera:

```
dist/
```

Puede desplegarse en:

- Vercel
- Netlify
- Firebase Hosting
- AWS Amplify
- GitHub Pages
- Servidores Nginx o Apache

Fundamentos Teóricos de Vue 3 + TypeScript (Vite)

Vue 3 es un framework progresivo para construir interfaces de usuario. Combina la simplicidad de React con la estructura de Angular, ofreciendo una curva de aprendizaje suave y una arquitectura poderosa. Su punto más fuerte es la Composition API, un sistema declarativo de reactividad que permite separar la lógica y reutilizarla fácilmente. Vue es ampliamente usado en empresas por su facilidad, escalabilidad y su excelente integración con TypeScript.

1. Filosofía y arquitectura de Vue

Vue se define como un framework progresivo, porque:

- Puedes usarlo para una sola vista HTML.
- O puedes construir un SPA completo.
- O puedes escalarlo a arquitectura modular empresarial.

Su arquitectura base incluye:

- Componentes
- Reactive system (Composition API)
- Directivas reactivas (`v-if`, `v-for`, `v-model`)
- Router opcional (Vue Router)
- Manejo de estado opcional (Pinia)
- Single File Components (SFC)

Tabla 1. Pilares fundamentales de Vue 3

Pilar	Descripción	Ejemplo
SFC	Un solo archivo `.vue` con `<template>`, `<script>`, `<style>`	`Home.vue`
Reactividad automática	`ref()` y `reactive()` actualizan la vista	`const x = ref(0)`
Directivas	Declaraciones HTML mejoradas	`v-if`, `v-for`, `v-bind`
Composition API	Funciones reactivas para separar lógica	`setup() { const count = ref(0) }`
Templates declarativos	HTML reactivo sin JSX	`<div>{{ count }}</div>`

2. Flujo interno de Vue



3. Single File Components (SFC)

Es el formato principal:

```
<template>
  <h1>{{ mensaje }}</h1>
</template>

<script setup lang="ts">
import { ref } from 'vue';
const mensaje = ref("Hola Vue 3");
</script>

<style scoped>
h1 { color: blue; }
</style>
```

Cada archivo `.vue` contiene:

- **Template:** la vista
- **Script:** lógica o Composition API
- **Styles:** estilos locales

Tabla 2. Estructura de un SFC

Sección	Función
`<template>`	Vista declarativa
`<script>` / `<script setup>`	Lógica del componente
`<style>`	Estilos locales
`<style scoped>`	Aísla estilos por componente

4. Composition API

Vue 3 adopta un nuevo modelo reactivo basado en funciones:

4.1 `ref()`

Define un valor reactivo:

```
const contador = ref(0);
contador.value++;
```

4.2 `reactive()`

Define un objeto completo reactivo:

```
const producto = reactive({
  nombre: "Café",
  precio: 10.5
});
```

4.3 `computed()`

Valores derivados:

```
const precioConIVA = computed(() => producto.precio * 1.12);
```

4.4 `watch()`

Observa cambios:

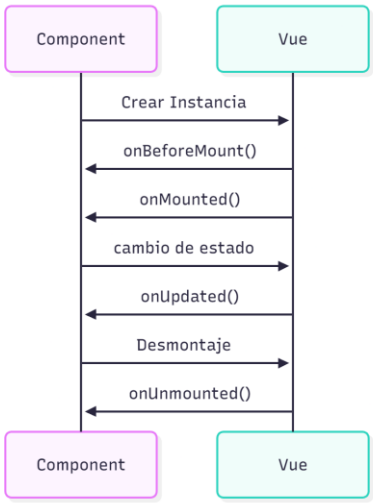
```
watch(contador, (nuevo) => console.log(nuevo));
```

5. Ciclo de vida de Vue

Vue ofrece hooks que pueden ejecutarse dentro de `script setup`:

Hook	Descripción
`onMounted`	Al montar el componente
`onUpdated`	Cuando la UI cambia
`onUnmounted`	Limpieza
`onBeforeMount`, `onBeforeUpdate`	Hooks avanzados

Diagrama:



6. Directivas de Vue

Vue tiene un conjunto pequeño pero poderoso de directivas:

Directiva	Uso
`v-if`	Mostrar/ocultar elementos
`v-for`	Iteración
`v-model`	Two-way binding
`v-bind` o `:`	Binding de atributos
`v-on` o `@`	Eventos

Ejemplos:

```
<p v-if="cargando">Cargando...</p>
<tr v-for="p in productos" :key="p.id">
  {{ p.nombre }}
</tr>
<input v-model="nombre">
<button @click="guardar">Guardar</button>
```

7. Reactividad automática

La joya de Vue es su sistema reactivo:

```
const nombre = ref("Café");
nombre.value = "Latte"; // se actualiza el DOM automáticamente
```

El motor reactivo de Vue detecta los accesos y actualiza el Virtual DOM eficientemente.

8. Routing en Vue (Vue Router)

Declaración del router:

```
const routes = [
  { path: '/', component: ListarProductos },
  { path: '/crear', component: CrearProducto },
  { path: '/editar/:id', component: EditarProducto }
];
```

Navegación:

```
router.push('/crear');
```

Parámetros:

```
const route = useRoute();
const id = route.params.id;
```

9. Componentes y comunicación

9.1 Props (datos que entran al componente)

```
<script setup lang="ts">
defineProps<{ producto: Producto }>();
</script>
```

9.2 Emits (eventos hacia el padre)

```
<script setup>
const emit = defineEmits(['guardar']);
emit('guardar', producto);
</script>
```

10. Manejo de estado global (Pinia)

Pinia es el sucesor oficial de Vuex:

```
export const useProductosStore = defineStore('productos', {
  state: () => ({
    listado: [] as Producto[]
  }),
  actions: {
    agregar(prod) {
      this.listado.push(prod);
    }
  }
});
```

11. Llamadas HTTP en Vue

Vue no trae módulo HTTP, se usa:

- `fetch()`
- axios
- servicios personalizados

Ejemplo:

```
const res = await fetch('/api/productos');
const data = await res.json();
```

12. Comparación Template vs JSX

Vue Template	React JSX
`<h1>{{ mensaje }}</h1>`	`<h1>{mensaje}</h1>`
HTML real con directivas	HTML-like dentro de JS
Más claro para principiantes	Más poderoso para lógica compleja

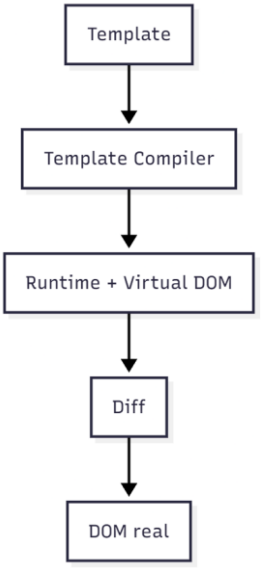
13. Organización de carpetas

Estructura recomendada:

```
src/
  features/
  services/
  models/
  router/
  App.vue
  main.ts
```

14. Virtual DOM en Vue

Vue utiliza un Virtual DOM optimizado con “compiler hinting”:



Ventajas:

- Cambios mínimos sobre el DOM
- Eficiencia en actualizaciones
- Muy buena performance

15. Ventajas de Vue

Ventaja	Razón
Curva de aprendizaje suave	Templates muy intuitivos
Reactividad sencilla	`ref()` y `reactive()` fáciles de entender
Excelente documentación	Didáctica y clara
Performance alta	Optimización por defecto
Ecosistema oficial	Vue Router, Pinia, Vite
Alto nivel de adopción profesional	Asia, Europa y LATAM

16. Desventajas de Vue

Desventaja	Explicación
Comunidad más pequeña que React	Menos librerías externas
Diferencias entre Options y Composition API	Puede confundir
Menos usado en grandes empresas USA	Aun así fuerte globalmente

17. Despliegue

Compilación con Vite:

```
npm run build
```

Salida:

```
dist/
```

Plataformas recomendadas:

- Vercel
- Netlify
- Firebase Hosting
- Azure Static Web Apps
- Nginx / Apache